

DE Full Report

Sam Remmey, James Siegener, Jacob Turner

May 2, 2025

Differential Expression Analysis

We've conducted DE analyses on both the high confidence 6-sample subset (3 POAG, 3 cadaver) as well as an 11-sample subset we formed by combining the 3 high confidence cadaver control samples with the 8 low confidence POAG samples. This report contains results of both series of analyses as well as a list of extracted genes that were found to be differentially expressed in both subsets, followed by functional enrichment analysis conducted on this extracted set of genes.

Global variables:

```
# Threshold for significance: p-value of 5e-10 is commonly used
p_val <- 5e-10
```

DE: 6 samples

```
gene_6 <- readxl::read_excel("gene_count_6.xls")

counts_6 <- gene_6 %>% select(gene_id, OS0658, OS21014, OD21014, AD22, RNA35, RNA62)

counts_6 <- counts_6 %>% column_to_rownames(var = "gene_id")

counts_6 <- as.matrix(counts_6)

mode(counts_6) <- "numeric"

sample_info_6 <- data.frame(
  row.names = colnames(counts), # Sample names
  condition = c("control", "control", "control", "glaucoma", "glaucoma", "glaucoma") # Group labels
)

sample_info_6$condition <- factor(sample_info_6$condition, levels = c("control", "glaucoma"))
```

```
dds_6 <- DESeqDataSetFromMatrix(countData = counts_6,
                                colData = sample_info_6,
                                design = ~ condition)
```

```
## converting counts to integer mode
```

```
# Filter out low-expression genes
# dds_6 <- dds_6[rowSums(counts_6(dds_6)) > 10, ]

# Run DESeq2 normalization and differential expression analysis
dds_6 <- DESeq(dds_6)
```

```
## estimating size factors
```

```
## estimating dispersions
```

```
## gene-wise dispersion estimates
```

```
## mean-dispersion relationship
```

```
## final dispersion estimates
```

```
## fitting model and testing
```

```
# Extract results (glaucoma vs. control)
res_6 <- results(dds_6, contrast = c("condition", "glaucoma", "control"))

# View summary
summary(res_6)
```

```
##
## out of 46831 with nonzero total read count
## adjusted p-value < 0.1
## LFC > 0 (up)      : 1271, 2.7%
## LFC < 0 (down)    : 1138, 2.4%
## outliers [1]      : 117, 0.25%
## low counts [2]     : 25082, 54%
## (mean count < 3)
## [1] see 'cooksCutoff' argument of ?results
## [2] see 'independentFiltering' argument of ?results
```

```
## Extract significant genes
sig_genes_6_adj <- subset(res_6, pvalue < p_val & abs(log2FoldChange) > 1)
all_genes_6_adj <- rownames(sig_genes_6_adj)
length(all_genes_6_adj)
```

```
## [1] 64
```

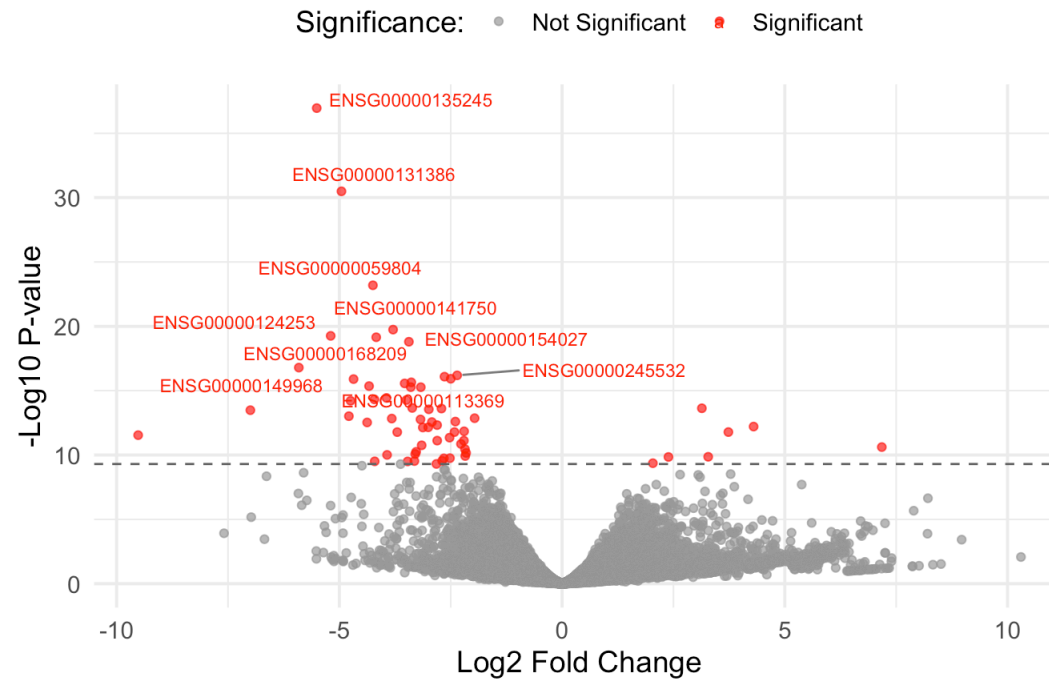
```
logfc_thresh <- 1 # threshold for large fold change

# Prepare data
res_df_6 <- as.data.frame(res_6) %>%
  rownames_to_column(var = "gene") %>%
  drop_na() %>%
  mutate(
    log2FoldChange = as.numeric(log2FoldChange),
    padj = as.numeric(padj),
    pvalue = as.numeric(pvalue),
    neg_log10_pvalue = -log10(pvalue),
    sig = case_when(
      pvalue < p_val ~ "Significant", # using pvalue rather than padj because our manually set p_val is very str
ict
      TRUE ~ "Not Significant"))

# Label the top genes
topp_genes <- res_df_6 %>%
  filter(pvalue < p_val) %>%
  top_n(-10, pvalue) # 10 genes with smallest p-values

# Volcano plot
ggplot(res_df_6, aes(x = log2FoldChange, y = neg_log10_pvalue, color = sig)) +
  geom_point(alpha = 0.7, size = 1.5) +
  scale_color_manual(values = c(
    "Significant" = "red",
    "Not Significant" = "grey60")) +
  geom_text_repel(
    data = topp_genes,
    aes(label = gene),
    size = 3,
    max.overlaps = 10,
    box.padding = 0.3,
    point.padding = 0.2,
    segment.color = 'grey50') +
  geom_hline(yintercept = -log10(p_val), linetype = "dashed", color = "gray40") +
  theme_minimal(base_size = 14) +
  labs(
    title = "Volcano Plot of Differential Gene Expression: 6 Samples",
    x = "Log2 Fold Change",
    y = "-Log10 P-value",
    color = "Significance:") +
  theme(
    plot.title = element_text(face = "bold", hjust = 0.5),
    legend.position = "top")
```

Volcano Plot of Differential Gene Expression: 6 Samples



```
# Ensure top genes match the volcano plot
topp_genes_6 <- topp_genes$gene # pulled from volcano plot step

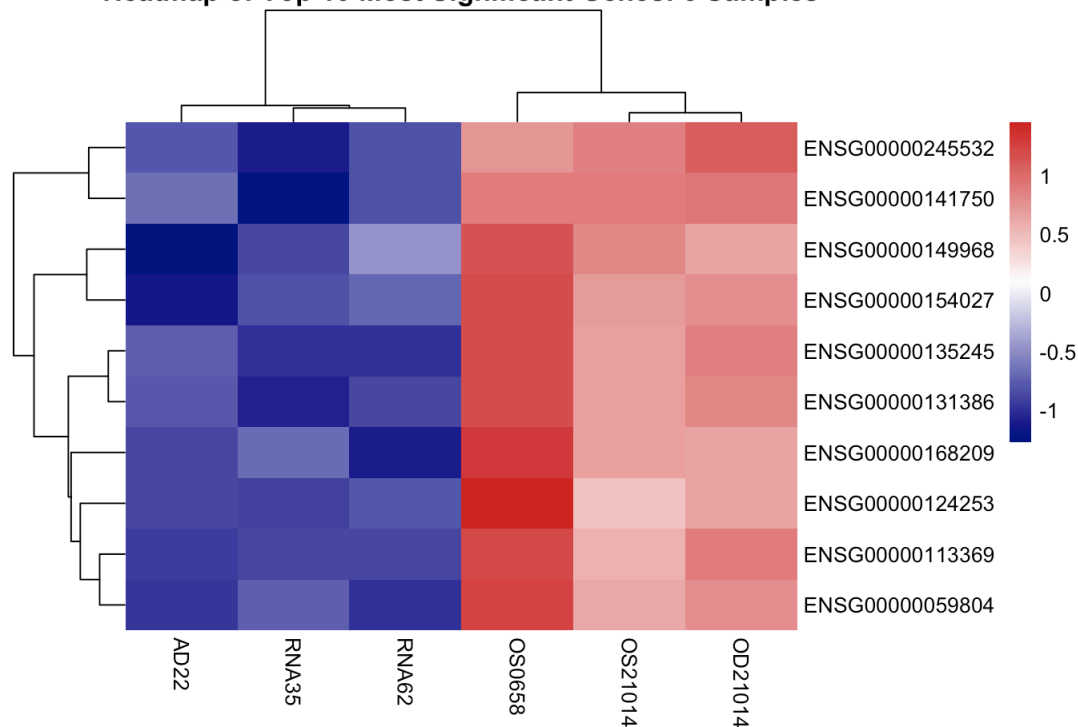
# Extract normalized expression matrix
vsd_6 <- vst(dds_6, blind = FALSE) # DESeq2 object → variance-stabilized matrix
vsd_mat <- assay(vsd_6)

# Subset matrix for top genes
heatmap_data <- vsd_mat[topp_genes_6, ]

# Z-score normalize across samples for better visual contrast
heatmap_scaled <- t(scale(t(heatmap_data)))

# Build heatmap with labels and title
pheatmap(
  mat = heatmap_scaled,
  cluster_rows = TRUE,
  cluster_cols = TRUE,
  color = colorRampPalette(c("navy", "white", "firebrick3"))(100),
  main = "Heatmap of Top 10 Most Significant Genes: 6 Samples",
  fontsize_row = 10,
  fontsize_col = 10,
  border_color = NA,
  show_rownames = TRUE,
  show_colnames = TRUE,
  labels_row = topp_genes_6)
```

Heatmap of Top 10 Most Significant Genes: 6 Samples



```
# PCA
```

```
# Run PCA
```

```
pcaData <- plotPCA(vsd_6, intgroup = "condition", returnData = TRUE)
```

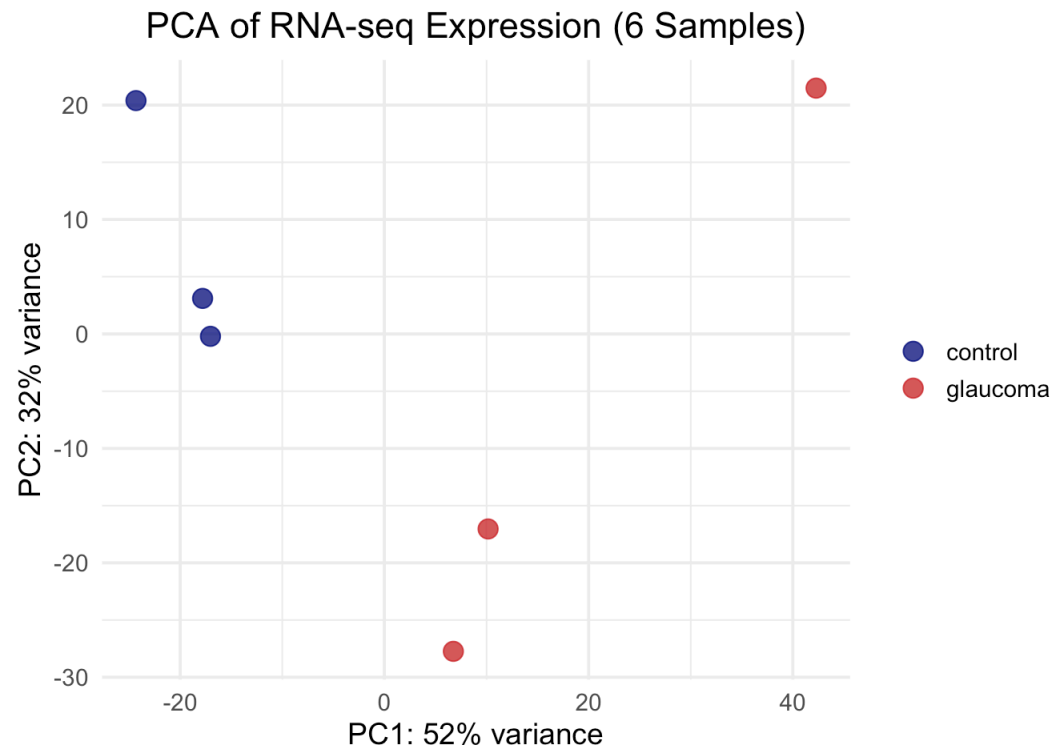
```
## using ntop=500 top features by variance
```

```
# Calculate % variance explained by PC1 and PC2
```

```
percentVar <- round(100 * attr(pcaData, "percentVar"))
```

```
# Plot
```

```
ggplot(pcaData, aes(x = PC1, y = PC2, color = condition)) +
  geom_point(size = 4, alpha = 0.8) +
  xlab(paste0("PC1: ", percentVar[1], "% variance")) +
  ylab(paste0("PC2: ", percentVar[2], "% variance")) +
  ggtitle("PCA of RNA-seq Expression (6 Samples)") +
  theme_minimal(base_size = 14) +
  scale_color_manual(values = c("glaucoma" = "firebrick3", "control" = "navy")) +
  theme(
    plot.title = element_text(hjust = 0.5),
    legend.title = element_blank())
```



DE: 11 samples

```
gene_16 <- data.frame(readxl::read_excel("gene_count_16.xls"))

counts_matrix <- gene_16 %>%
  select(gene_id, OS0658, OS21014, OD21014, AD24, AD25, RNA41, RNA45, RNA47, RNA48, RNA53, RNA44)
rownames(counts_matrix) <- counts_matrix$gene_id
counts_matrix <- counts_matrix %>%
  select(-gene_id)
counts_matrix <- as.matrix(counts_matrix) # Convert to numeric

samples <- c('OS0658', 'OS21014', 'OD21014', 'AD24', 'AD25', 'RNA41', 'RNA45', 'RNA47', 'RNA48', 'RNA53', 'RNA44')
label <- c('control', 'control', 'control', 'glaucoma', 'glaucoma', 'glaucoma', 'glaucoma', 'glaucoma', 'glaucoma', 'glaucoma', 'glaucoma')
labels <- data.frame(samples, label)
rownames(labels) <- labels$samples
labels <- labels %>%
  select(-samples)
```

```
## Create a DESeqDataSet object
dds <- DESeqDataSetFromMatrix(countData = counts_matrix,
                              colData = labels,
                              design = ~ label)
```

```
## converting counts to integer mode
```

```
## Warning in DESeqDataSet(se, design = design, ignoreRank): some variables in
## design formula are characters, converting to factors
```

```
# Filter low-count genes (optional)
#dds <- dds[rowSums(counts(dds)) > 10, ]

## Run DESeq analysis
dds <- DESeq(dds)
```

```
## estimating size factors
```

```
## estimating dispersions
```

```
## gene-wise dispersion estimates
```

```
## mean-dispersion relationship
```

```
## final dispersion estimates
```

```
## fitting model and testing
```

```
## -- replacing outliers and refitting for 98 genes
## -- DESeq argument 'minReplicatesForReplace' = 7
## -- original counts are preserved in counts(dds)
```

```
## estimating dispersions
```

```
## fitting model and testing
```

```
res <- results(dds)
```

```
# p values
res <- res[order(res$pvalue), ]
head(res)
```

```
## log2 fold change (MLE): label glaucoma vs control
## Wald test p-value: label glaucoma vs control
## DataFrame with 6 rows and 6 columns
##           baseMean log2FoldChange      lfcSE      stat      pvalue
##           <numeric>      <numeric> <numeric> <numeric> <numeric>
## ENSG00000281344    776.229      8.75541  0.465010   18.8284 4.41535e-79
## ENSG00000135245    885.723     -6.37560  0.363360   -17.5462 6.35477e-69
## ENSG00000170345   1157.796     -5.69342  0.354089   -16.0791 3.57819e-58
## ENSG00000215483    183.750      5.58638  0.353037    15.8238 2.13257e-56
## ENSG00000226674    158.431      6.48094  0.451665    14.3490 1.08068e-46
## ENSG00000198838    402.766      7.09310  0.508280    13.9551 2.92821e-44
##                padj
##                <numeric>
## ENSG00000281344 2.04815e-74
## ENSG00000135245 1.47389e-64
## ENSG00000170345 5.53272e-54
## ENSG00000215483 2.47309e-52
## ENSG00000226674 1.00259e-42
## ENSG00000198838 2.26385e-40
```

```
# Extract significant genes
sig_genes_11_adj <- subset(res, pvalue < p_val & abs(log2FoldChange) > 1)
all_genes_11_adj <- rownames(sig_genes_11_adj)
```

```
# Compare number of significant genes from 11-sample vs 6-sample
length(all_genes_11_adj)
```

```
## [1] 4968
```

```
length(all_genes_6_adj)
```

```
## [1] 64
```

Note the huge difference in number of significant genes here. This result suggests expression in the 11-sample subset may contain a lot of noise.

```

logfc_thresh <- 1 # threshold for large fold change

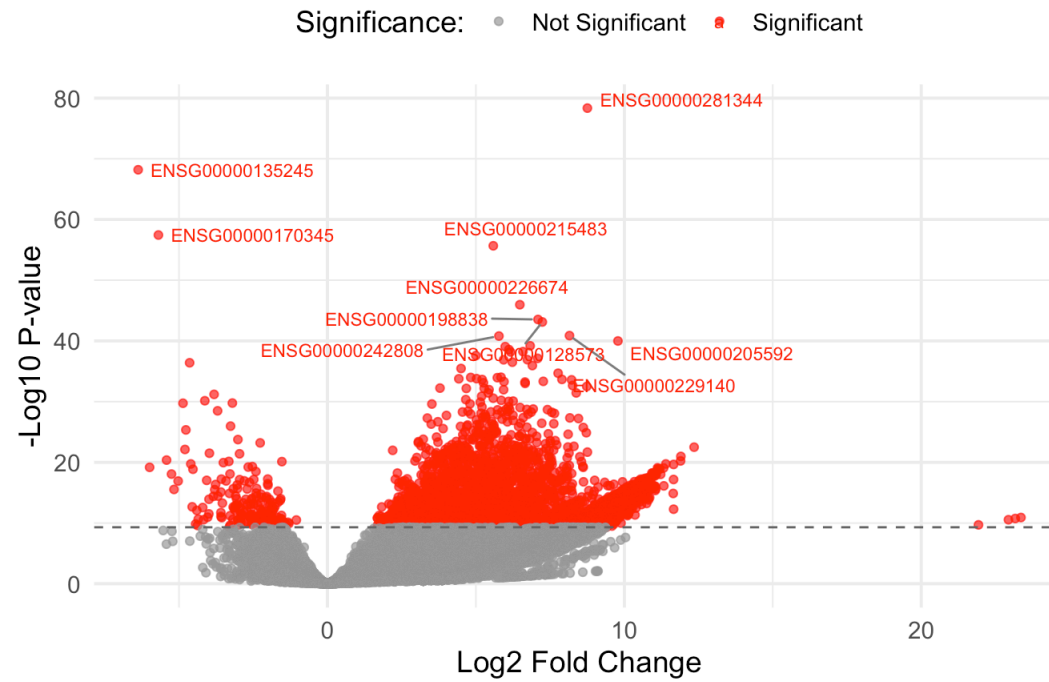
# Prepare data
res_df_11 <- as.data.frame(res) %>%
  rownames_to_column(var = "gene") %>%
  drop_na() %>%
  mutate(
    log2FoldChange = as.numeric(log2FoldChange),
    padj = as.numeric(padj),
    pvalue = as.numeric(pvalue),
    neg_log10_pvalue = -log10(pvalue),
    sig = case_when(
      pvalue < p_val ~ "Significant", # using pvalue rather than padj because our manually set p_val is very str
ict
      TRUE ~ "Not Significant"))

# Label the top genes
topp_genes11 <- res_df_11 %>%
  filter(pvalue < p_val) %>%
  top_n(-10, pvalue) # 10 genes with smallest p-values

# Volcano plot
ggplot(res_df_11, aes(x = log2FoldChange, y = neg_log10_pvalue, color = sig)) +
  geom_point(alpha = 0.7, size = 1.5) +
  scale_color_manual(values = c(
    "Significant" = "red",
    "Not Significant" = "grey60")) +
  geom_text_repel(
    data = topp_genes11,
    aes(label = gene),
    size = 3,
    max.overlaps = 20,
    box.padding = 0.3,
    point.padding = 0.2,
    segment.color = 'grey50') +
  geom_hline(yintercept = -log10(p_val), linetype = "dashed", color = "gray40") +
  theme_minimal(base_size = 14) +
  labs(
    title = "Volcano Plot of Differential Gene Expression: 11 Samples",
    x = "Log2 Fold Change",
    y = "-Log10 P-value",
    color = "Significance:") +
  theme(
    plot.title = element_text(face = "bold", hjust = 0.5),
    legend.position = "top")

```

Volcano Plot of Differential Gene Expression: 11 Samples



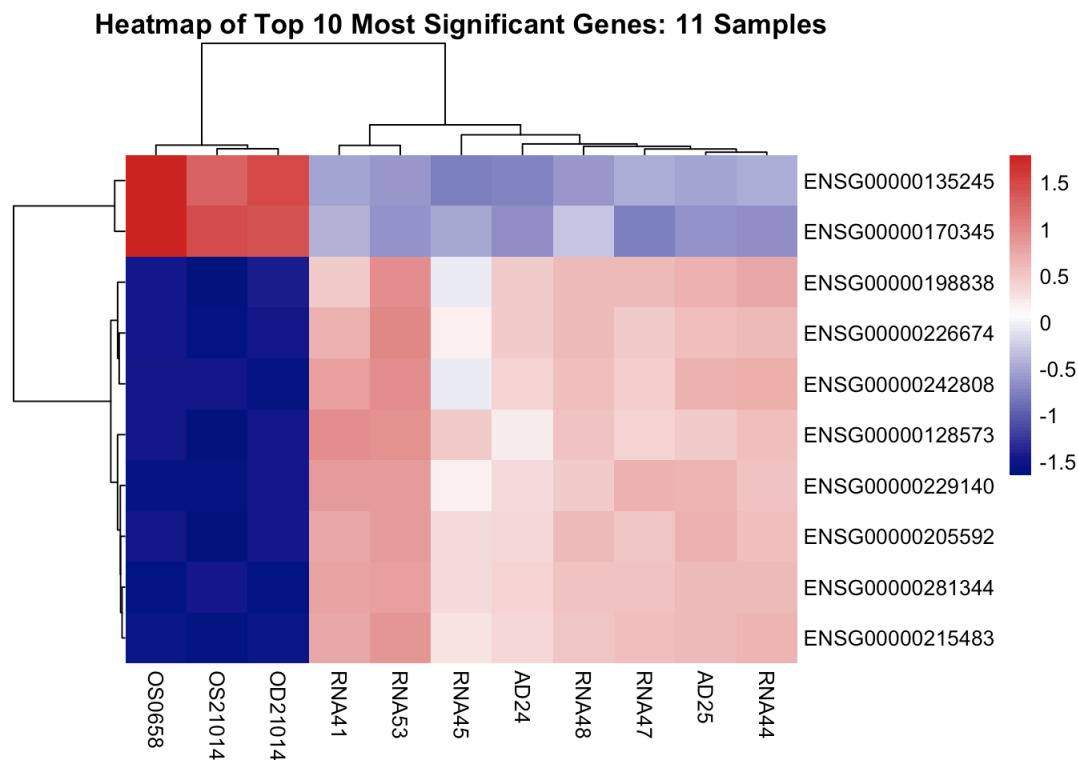
```
# Ensure top genes match the volcano plot
topp_genes_11 <- topp_genes11$gene # pulled from volcano plot step

# Extract normalized expression matrix
vsd_11 <- vst(dds, blind = FALSE) # DESeq2 object → variance-stabilized matrix
vsd_mat11 <- assay(vsd_11)

# Subset matrix for top genes
heatmap_data11 <- vsd_mat11[topp_genes_11, ]

# Z-score normalize across samples for better visual contrast
heatmap_scaled11 <- t(scale(t(heatmap_data11)))

# Build heatmap with labels and title
pheatmap(
  mat = heatmap_scaled11,
  cluster_rows = TRUE,
  cluster_cols = TRUE,
  color = colorRampPalette(c("navy", "white", "firebrick3"))(100),
  main = "Heatmap of Top 10 Most Significant Genes: 11 Samples",
  fontsize_row = 10,
  fontsize_col = 10,
  border_color = NA,
  show_rownames = TRUE,
  show_colnames = TRUE,
  labels_row = topp_genes_11)
```



```
# PCA
```

```
# Run PCA
```

```
pcaData11 <- plotPCA(vsd_11, intgroup = "label", returnData = TRUE)
```

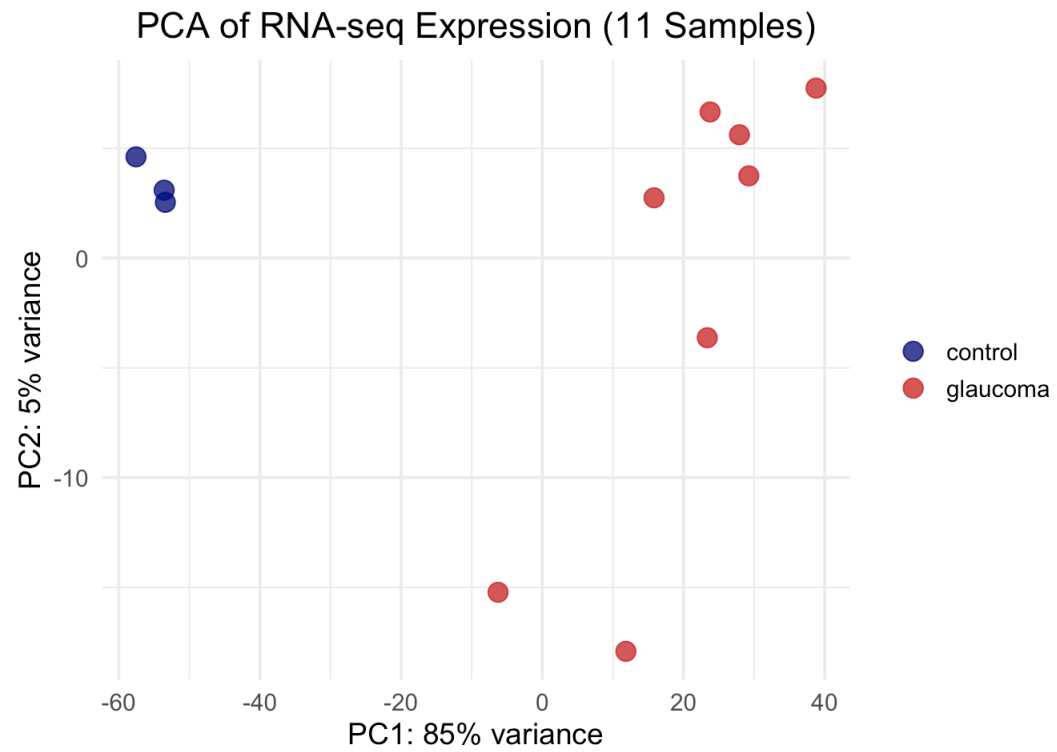
```
## using ntop=500 top features by variance
```

```
# Calculate % variance explained by PC1 and PC2
```

```
percentVar <- round(100 * attr(pcaData11, "percentVar"))
```

```
# Plot
```

```
ggplot(pcaData11, aes(x = PC1, y = PC2, color = label)) +
  geom_point(size = 4, alpha = 0.8) +
  xlab(paste0("PC1: ", percentVar[1], "% variance")) +
  ylab(paste0("PC2: ", percentVar[2], "% variance")) +
  ggtitle("PCA of RNA-seq Expression (11 Samples)") +
  theme_minimal(base_size = 14) +
  scale_color_manual(values = c("glaucoma" = "firebrick3", "control" = "navy")) +
  theme(
    plot.title = element_text(hjust = 0.5),
    legend.title = element_blank())
```



PCA again: this time add back the 3 high confidence POAG samples and run PCA on that.

```
gene_16 <- data.frame(readxl::read_excel("gene_count_16.xls"))

counts_matrix2 <- gene_16 %>%
  select(gene_id, OS0658, OS21014, OD21014, AD22, RNA35, RNA62, AD24, AD25, RNA41, RNA45, RNA47, RNA48, RNA53, RNA44)
rownames(counts_matrix2) <- counts_matrix2$gene_id
counts_matrix2 <- counts_matrix2 %>%
  select(-gene_id)
counts_matrix2 <- as.matrix(counts_matrix2) # Convert to numeric

samples <- c('OS0658','OS21014','OD21014', 'AD22', 'RNA35', 'RNA62', 'AD24','AD25','RNA41','RNA45','RNA47','RNA48','RNA53','RNA44')
label <- c('control','control','control','POAG_HighConf','POAG_HighConf','POAG_HighConf','POAG_LowConf','POAG_LowConf','POAG_LowConf','POAG_LowConf','POAG_LowConf','POAG_LowConf','POAG_LowConf','POAG_LowConf')
labels <- data.frame(samples, label)
rownames(labels) <- labels$samples
labels <- labels %>%
  select(-samples)

## Create a DESeqDataSet object
dds2 <- DESeqDataSetFromMatrix(countData = counts_matrix2,
                              colData = labels,
                              design = ~ label)
```

```
## converting counts to integer mode
```

```
## Warning in DESeqDataSet(se, design = design, ignoreRank): some variables in
## design formula are characters, converting to factors
```

```
# Filter low-count genes (optional)
#dds <- dds[rowSums(counts(dds)) > 10, ]
```

```
## Run DESeq analysis
dds2 <- DESeq(dds2)
```

```
## estimating size factors
```

```
## estimating dispersions
```

```
## gene-wise dispersion estimates
```

```
## mean-dispersion relationship
```

```
## final dispersion estimates
```

```
## fitting model and testing
```

```
## -- replacing outliers and refitting for 123 genes  
## -- DESeq argument 'minReplicatesForReplace' = 7  
## -- original counts are preserved in counts(dds)
```

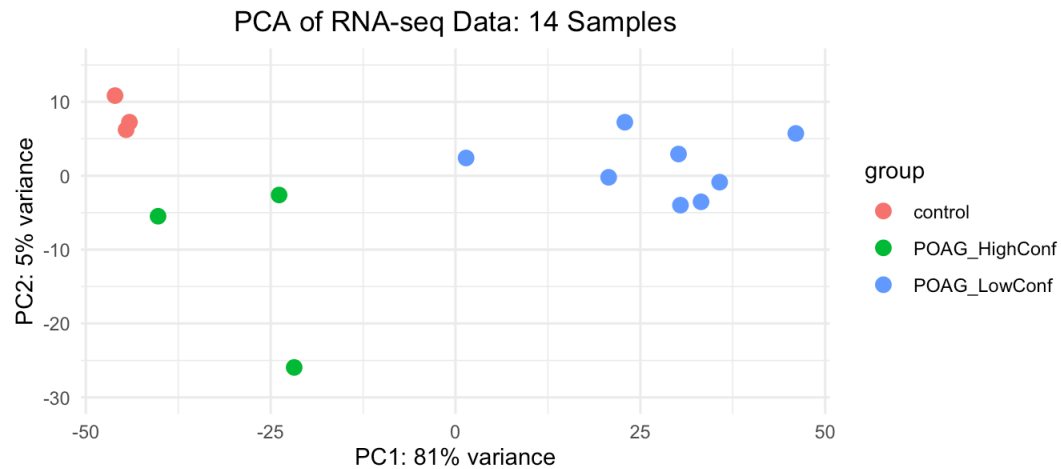
```
## estimating dispersions
```

```
## fitting model and testing
```

```
res2 <- results(dds2)
```

```
vsd_14 <- vst(dds2, blind = FALSE)  
  
plotPCA(vsd_14, intgroup = "label") +  
  theme_minimal() +  
  ggtitle("PCA of RNA-seq Data: 14 Samples") +  
  theme(plot.title = element_text(hjust = 0.5)) +  
  ylim(-30, 15)
```

```
## using ntop=500 top features by variance
```



Comparison

Identify overlap in differentially expressed genes.

```
# Overlap with strict p-value
overlap_adj <- intersect(all_genes_6_adj, all_genes_11_adj)
length(overlap_adj)
```

```
## [1] 32
```

Jaccard similarity

```
# Calculate the intersection and union of the sets
intersection <- length(overlap_adj)
union <- length(union(all_genes_6_adj, all_genes_11_adj))

# Calculate Jaccard Similarity
jaccard_similarity <- intersection / union
jaccard_similarity
```

```
## [1] 0.0064
```

From here, we will change the significance level for genes in the contaminated subset (make stricter) to yield fewer genes.

Maybe we want to find the p-value that maximizes Jaccard similarity? We want to determine if the same genes that are significant in the high-confidence set are also present at high significance in the contaminated set, just maybe buried under contamination/noise.

Use p-value of 5e-20 to extract significant genes, then calculate Jaccard similarity again:

```
p_val <- 5e-20

# Extract significant genes
sig_genes_11_adj2 <- subset(res, pvalue < p_val & abs(log2FoldChange) > 1)
all_genes_11_adj2 <- rownames(sig_genes_11_adj2)

# Get overlap
overlap_adj2 <- intersect(all_genes_6_adj, all_genes_11_adj2)
length(overlap_adj2)
```

```
## [1] 13
```

```
# Get Jaccard similarity
intersection <- length(overlap_adj2)
union <- length(union(all_genes_6_adj, all_genes_11_adj2))
jaccard_similarity <- intersection / union
jaccard_similarity
```

```
## [1] 0.02947846
```

Now we will iterate over several p-value thresholds.

```
p_value_list = c(5e-6, 5e-8, 5e-10, 5e-12, 5e-14, 5e-16, 5e-18, 5e-20, 5e-22, 5e-24, 5e-26)
jacc <- data.frame('p_val'=numeric(), 'length_overlap'=numeric(), 'jaccard_sim'=numeric())

for (p in p_value_list){
  sig_genes_11_adj <- subset(res, pvalue < p & abs(log2FoldChange) > 1)
  all_genes_11_adj <- rownames(sig_genes_11_adj)

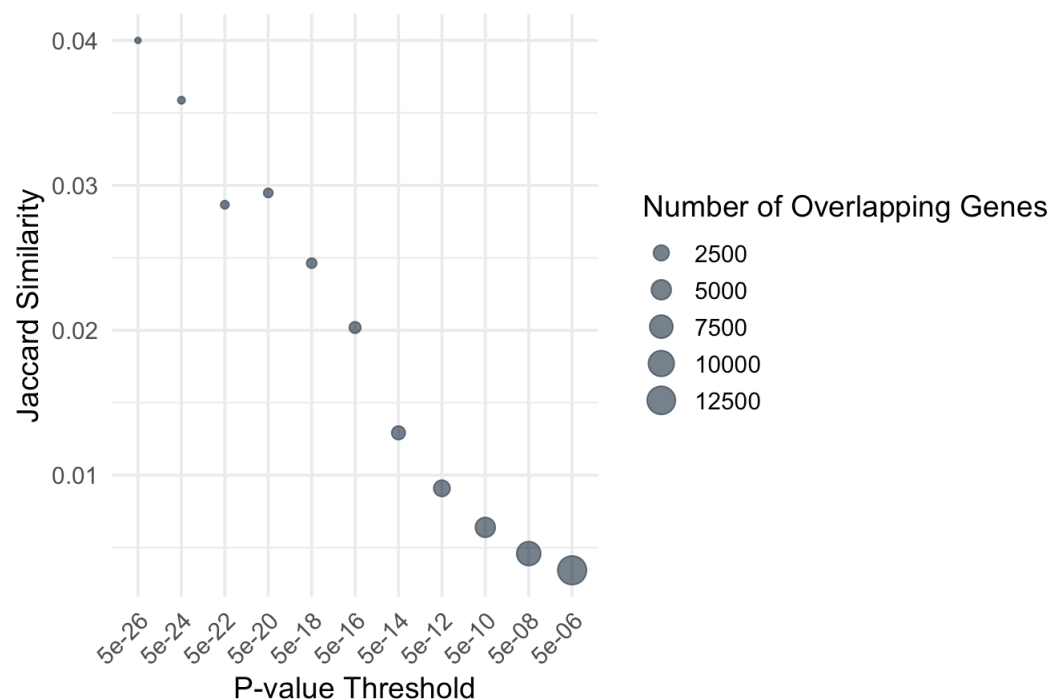
  overlap_adj <- intersect(all_genes_6_adj, all_genes_11_adj)

  intersection <- length(overlap_adj)
  union <- length(union(all_genes_6_adj, all_genes_11_adj))
  jaccard_similarity <- intersection / union

  new <- data.frame('p_val' = p, 'length_overlap'=union, 'jaccard_sim'=jaccard_similarity)
  jacc <- rbind(jacc, new)
}
```

```
jacc$p_val <- factor(jacc$p_val)
ggplot(jacc, aes(x = p_val, y = jaccard_sim, size = length_overlap)) +
  geom_point(color = "#2C3E50", alpha = 0.7) +
  scale_size_continuous(name = "Number of Overlapping Genes") +
  theme_minimal(base_size = 14) +
  labs(
    title = "Jaccard Similarity Across P-value Thresholds",
    x = "P-value Threshold",
    y = "Jaccard Similarity") +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"),
        axis.text.x = element_text(angle = 45, hjust = 1))
```

Jaccard Similarity Across P-value Thresholds



Make table of overlapping genes

```
df_res6 <- data.frame(res_6)
df_res11 <- data.frame(res)

# Convert row names to a column
df_res6$GENE <- rownames(df_res6)
df_res11$GENE <- rownames(df_res11)

# Merge
df_overlap <- merge(df_res6, df_res11, by = "GENE", all = FALSE) # Inner join but doesn't matter cause same in both

# Now filter so that it's only the 32 overlapping genes
df_overlap <- df_overlap[df_overlap$GENE %in% overlap_adj, ]

df_overlap <- df_overlap %>%
  dplyr::rename(baseMean.6 = baseMean.x, log2FoldChange.6 = log2FoldChange.x, lfcSE.6 = lfcSE.x, stat.6 = stat.x,
    pvalue.6 = pvalue.x,
    padj.6 = padj.x, baseMean.11 = baseMean.y, log2FoldChange.11 = log2FoldChange.y, lfcSE.11 = lfcSE.y, stat.11 = stat.y,
    pvalue.11 = pvalue.y, padj.11 = padj.y)
```

```
# Write to csv
#write.csv(df_overlap, 'GeneOverlap.csv', row.names = F)

# Get most significant genes (most significant in 6-sample)

df_overlap2 = df_overlap %>%
  dplyr::rename(EnsemblGeneID = GENE) %>%
  arrange(pvalue.6) %>%
  mutate(DirectionOfExpression = ifelse(log2FoldChange.6 > 0, 'Upregulated', 'Downregulated')) %>%
  select(EnsemblGeneID, DirectionOfExpression) %>%
  head(10)

kable(df_overlap2, caption = "Top 10 Differentially Expressed Genes Across High- and Low-Confidence Samples") %>%
  column_spec(1, width = "6cm", extra_css = "padding-left: 20px;") %>%
  column_spec(2, width = "6cm", extra_css = "padding-left: 20px;") %>%
  row_spec(0, extra_css = "padding-left: 20px; text-align: left;")
```

Top 10 Differentially Expressed Genes Across High- and Low-Confidence Samples

EnsemblGeneID	DirectionOfExpression
ENSG00000135245	Downregulated
ENSG00000131386	Downregulated
ENSG00000059804	Downregulated
ENSG00000124253	Downregulated
ENSG00000168209	Downregulated
ENSG00000154027	Downregulated
ENSG00000149968	Downregulated
ENSG00000113369	Downregulated
ENSG00000111859	Downregulated
ENSG00000165507	Downregulated

Another volcano plot:

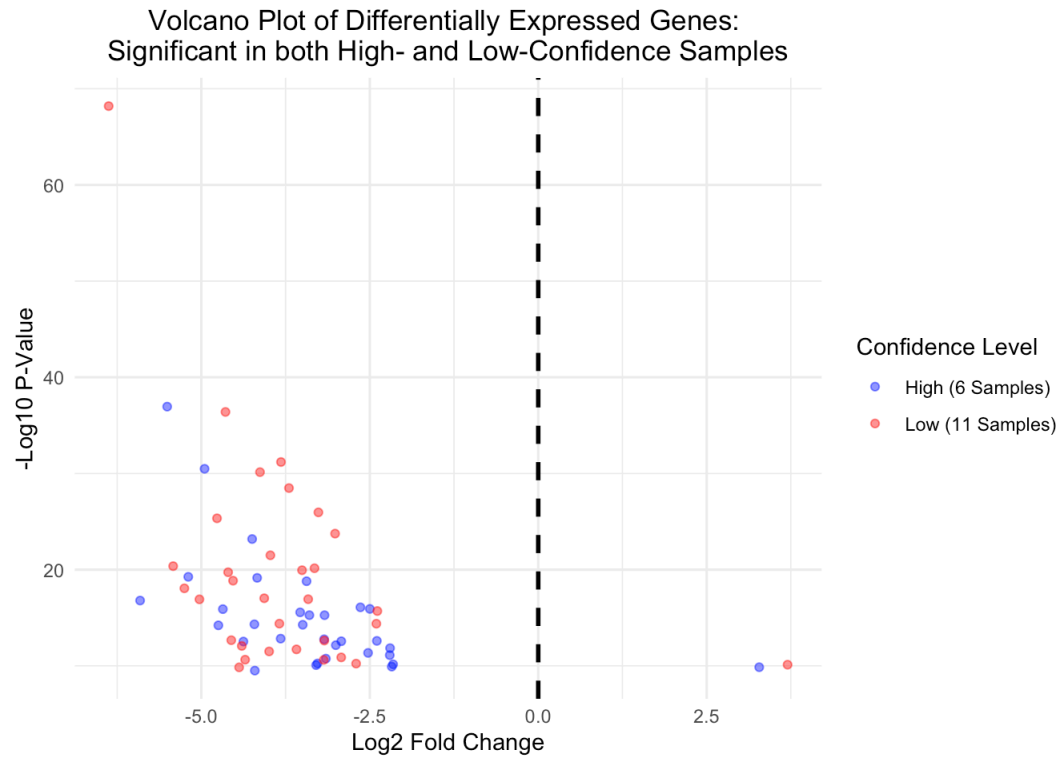
```
filtered_6 <- df_res6 %>% filter(GENE %in% overlap_adj)
filtered_11 <- df_res11 %>% filter(GENE %in% overlap_adj)
```

```
filtered_6$confidence <- "High (6 Samples)"
filtered_11$confidence <- "Low (11 Samples)"
```

```
overlap_data <- bind_rows(filtered_6, filtered_11)
```

```
overlap_data <- overlap_data %>%
  group_by(GENE) %>%
  filter(n() == 2) %>%
  filter(all(log2FoldChange > 0) | all(log2FoldChange < 0)) %>%
  ungroup()
```

```
ggplot(overlap_data, aes(x = log2FoldChange, y = -log10(pvalue), color = as.factor(confidence))) +
  geom_point(alpha = 0.5) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "black", linewidth = 1) +
  scale_color_manual(values = c("High (6 Samples)" = "blue", "Low (11 Samples)" = "red")) +
  theme_minimal() +
  xlab("Log2 Fold Change") +
  ylab("-Log10 P-Value") +
  ggtitle("Volcano Plot of Differentially Expressed Genes: \nSignificant in both High- and Low-Confidence Samples") +
  theme(plot.title = element_text(hjust = 0.5)) +
  labs(color = "Confidence Level")
```



The above plot contains a point for each gene that was found to be significantly differentially expressed in both the high- and low-confidence subsets. There are 32 of these genes in total.

Correlating DE results

```
dat <- read.csv('GeneOverlap.csv')
```

```
# Overlap - significant in both
cor(dat$baseMean.6, dat$baseMean.11) # 0.99
```

```
## [1] 0.9969782
```

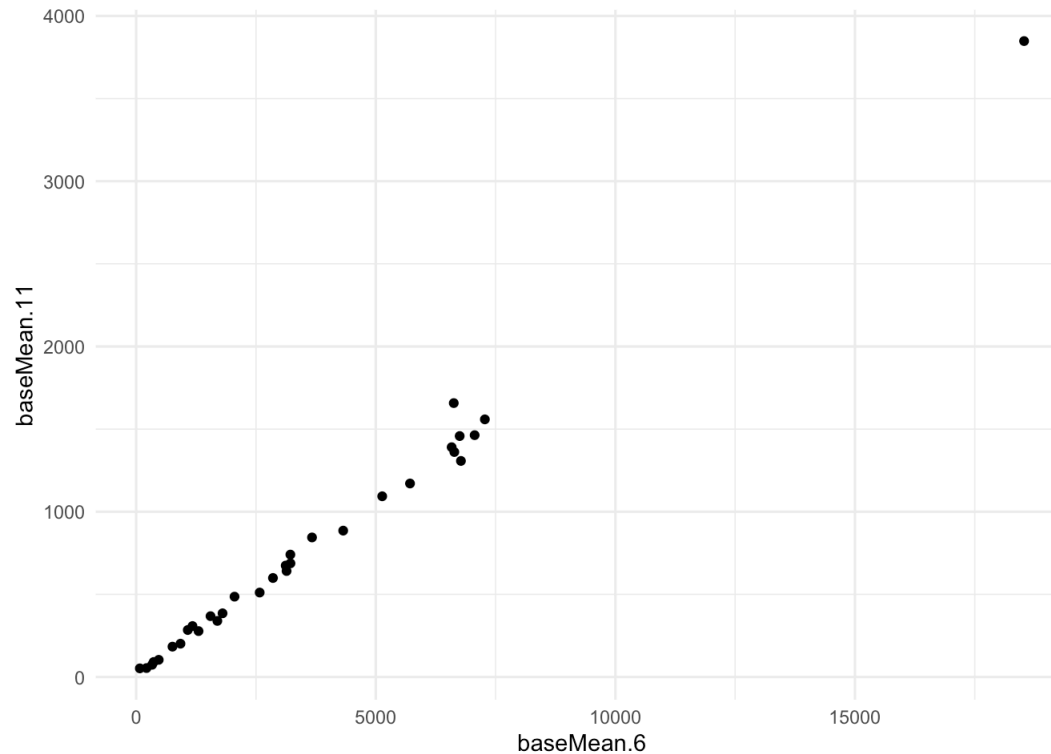
```
# All genes
cor(df_res6$baseMean, df_res11$baseMean) # 0.01
```

```
## [1] 0.01085744
```

```
# Only genes significantly differentially expressed in 6 samples  
only6_6 <- df_res6[rownames(df_res6) %in% all_genes_6_adj, ]  
only6_11 <- df_res11[rownames(df_res6) %in% all_genes_6_adj, ]  
cor(only6_6$baseMean, only6_11$baseMean) # 0.15
```

```
## [1] 0.1520028
```

```
# Genes significant across 6 samples and 11 samples  
ggplot(dat, aes(x=baseMean.6, y=baseMean.11))+geom_point()+theme_minimal()
```



Functional Enrichment Analysis

```
#install.packages(c("httr", "jsonlite", "curl"))  
#install.packages("gprofiler2")  
library(gprofiler2)
```

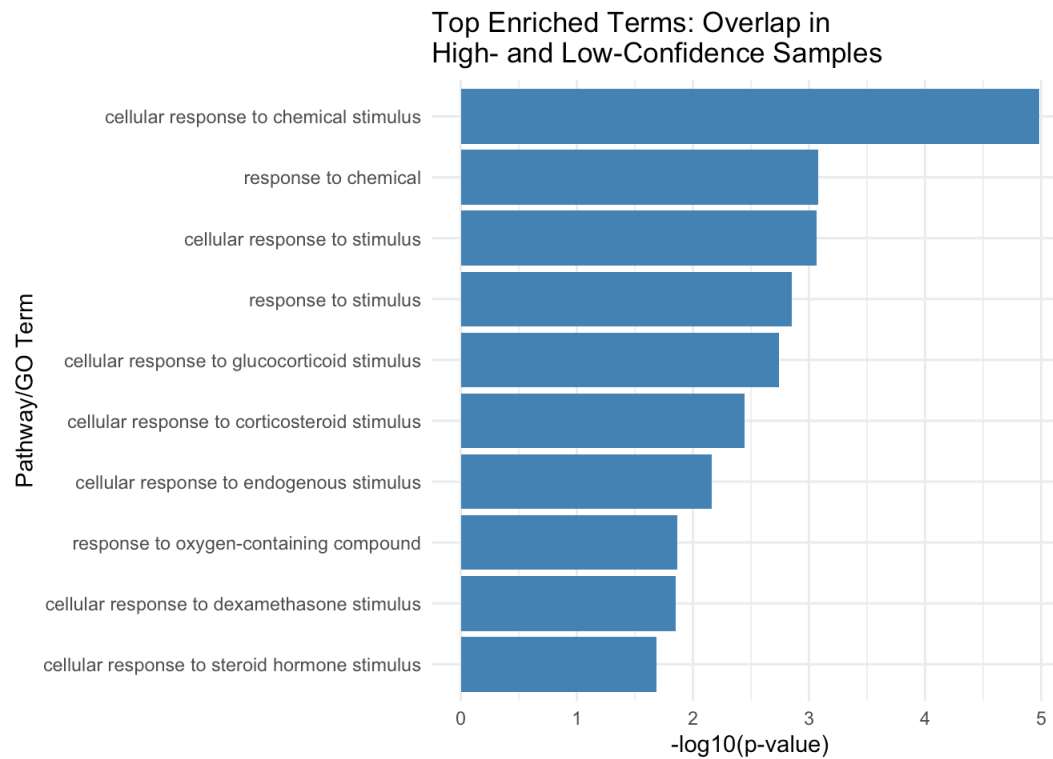
```
# Run functional enrichment analysis
gost_results <- gost(query = overlap_adj, # 32 overlapping genes
                    organism = "hsapiens", # Human genes
                    sources = c("GO:BP", "GO:MF", "GO:CC", "KEGG", "REACTOME"),
                    significant = TRUE)

# Print results
head(gost_results$result)
```

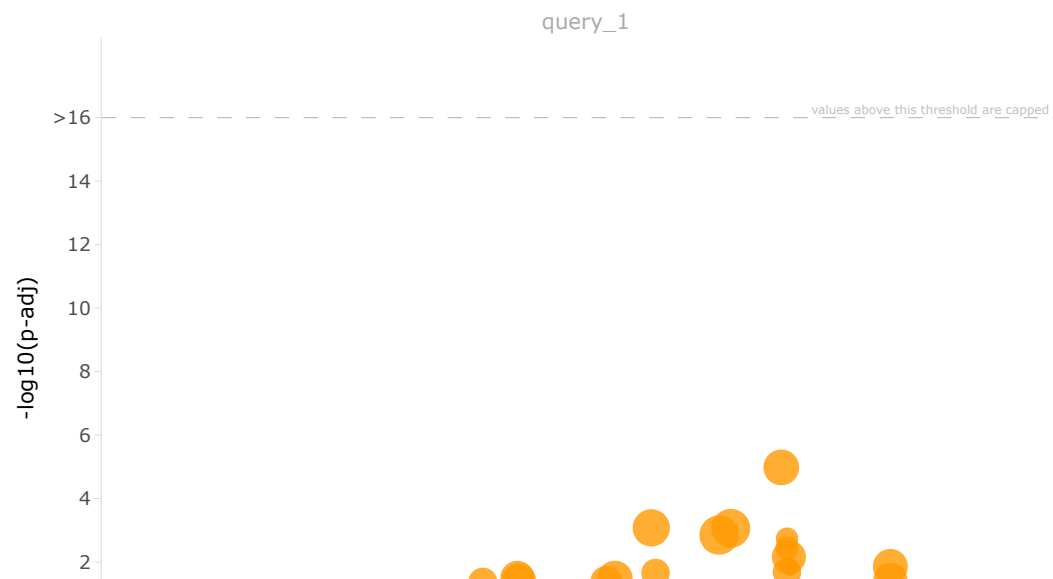
```
##      query significant      p_value term_size query_size intersection_size
## 1 query_1          TRUE 1.036936e-05      2186         30             16
## 2 query_1          TRUE 8.331266e-04      3867         30             18
## 3 query_1          TRUE 8.633581e-04      7320         30             24
## 4 query_1          TRUE 1.401140e-03      8993         30             26
## 5 query_1          TRUE 1.822305e-03         55         30              4
## 6 query_1          TRUE 3.581874e-03         65         30              4
##      precision      recall      term_id source
## 1 0.5333333 0.007319305 GO:0070887 GO:BP
## 2 0.6000000 0.004654771 GO:0042221 GO:BP
## 3 0.8000000 0.003278689 GO:0051716 GO:BP
## 4 0.8666667 0.002891138 GO:0050896 GO:BP
## 5 0.1333333 0.072727273 GO:0071385 GO:BP
## 6 0.1333333 0.061538462 GO:0071384 GO:BP
##
##      term_name effective_domain_size
## 1 cellular response to chemical stimulus 21017
## 2 response to chemical 21017
## 3 cellular response to stimulus 21017
## 4 response to stimulus 21017
## 5 cellular response to glucocorticoid stimulus 21017
## 6 cellular response to corticosteroid stimulus 21017
##      source_order      parents
## 1      16358 GO:00422....
## 2      10049 GO:0050896
## 3      13909 GO:00099....
## 4      13339 GO:0008150
## 5      16637 GO:00513....
## 6      16636 GO:00319....
```

```
# Extract top 10 enriched terms
top_terms <- gost_results$result[1:10,]

ggplot(top_terms, aes(x=reorder(term_name, -p_value), y=-log10(p_value))) +
  geom_bar(stat="identity", fill="steelblue") +
  coord_flip() +
  labs(title="Top Enriched Terms: Overlap in \nHigh- and Low-Confidence Samples", x="Pathway/GO Term", y="-log10
(p-value)") +
  theme_minimal()
```



```
# Interactive manhattan plot  
gostplot(gost_results, capped = TRUE, interactive = TRUE)
```





Now again with only genes significant in high confidence (6 sample) subset.

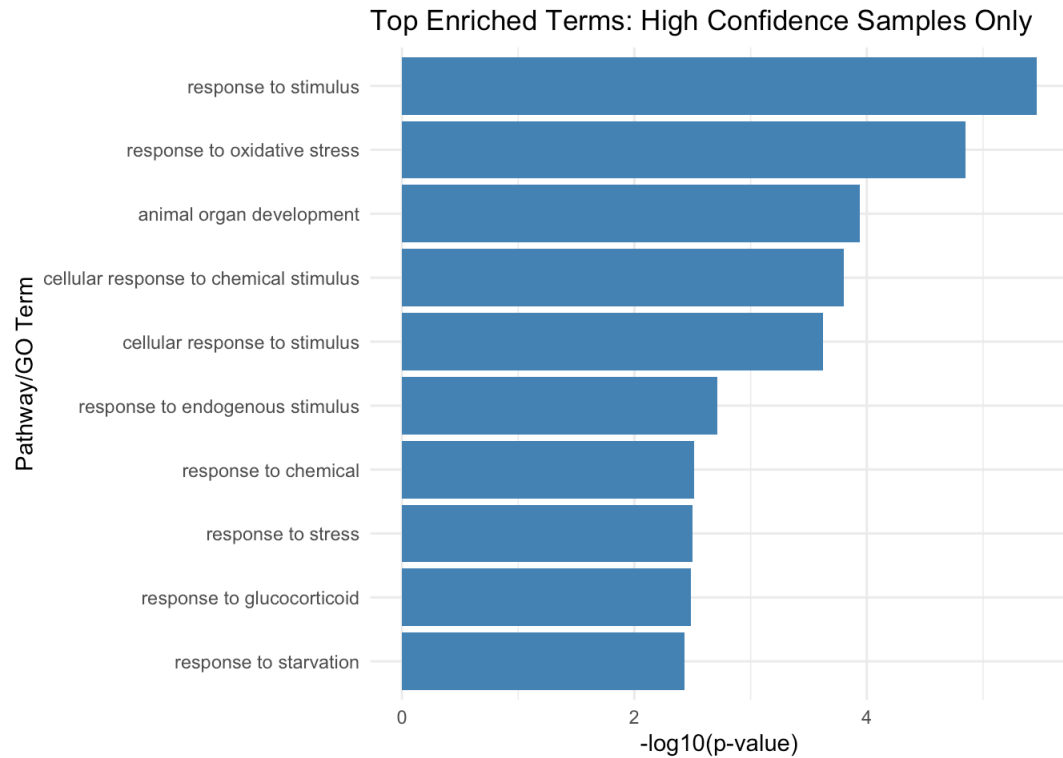
```
# Run functional enrichment analysis
gost_results2 <- gost(query = all_genes_6_adj, # 64 genes
                      organism = "hsapiens", # Human genes
                      sources = c("GO:BP", "GO:MF", "GO:CC", "KEGG", "REACTOME"),
                      significant = TRUE)

# Print results
head(gost_results2$result)
```

```
##      query significant      p_value term_size query_size intersection_size
## 1 query_1          TRUE 3.465601e-06    8993         56              46
## 2 query_1          TRUE 1.413877e-05     398         56              11
## 3 query_1          TRUE 1.153796e-04    3047         56              25
## 4 query_1          TRUE 1.584925e-04    2186         56              21
## 5 query_1          TRUE 2.380156e-04    7320         56              39
## 6 query_1          TRUE 1.937286e-03    1478         56              16
##      precision      recall   term_id source
## 1 0.8214286 0.005115090 GO:0050896 GO:BP
## 2 0.1964286 0.027638191 GO:0006979 GO:BP
## 3 0.4464286 0.008204792 GO:0048513 GO:BP
## 4 0.3750000 0.009606587 GO:0070887 GO:BP
## 5 0.6964286 0.005327869 GO:0051716 GO:BP
## 6 0.2857143 0.010825440 GO:0009719 GO:BP
##                                     term_name effective_domain_size source_order
## 1                                response to stimulus                21017        13339
## 2                response to oxidative stress                21017         2604
## 3                    animal organ development                21017        12798
## 4 cellular response to chemical stimulus                21017        16358
## 5                cellular response to stimulus                21017        13909
## 6                response to endogenous stimulus                21017         3617
##      parents
## 1 GO:0008150
## 2 GO:0006950
## 3 GO:0048856
## 4 GO:00422....
## 5 GO:00099....
## 6 GO:0050896
```

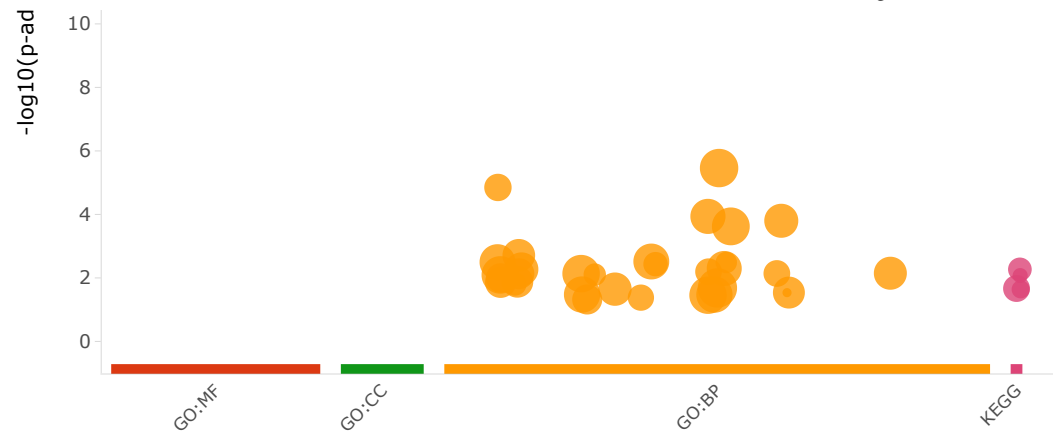
```
# Extract top 10 enriched terms
top_terms2 <- gost_results2$result[1:10,]

ggplot(top_terms2, aes(x=reorder(term_name, -p_value), y=-log10(p_value))) +
  geom_bar(stat="identity", fill="steelblue") +
  coord_flip() +
  labs(title="Top Enriched Terms: High Confidence Samples Only", x="Pathway/GO Term", y="-log10(p-value)") +
  theme_minimal()
```



```
# Interactive manhattan plot
gostplot(gost_results2, capped = TRUE, interactive = TRUE)
```





GWAS Comparison

```
library(org.Hs.eg.db)
```

```
## Loading required package: AnnotationDbi
```

```
##  
## Attaching package: 'AnnotationDbi'
```

```
## The following object is masked from 'package:dplyr':  
##  
## select
```

```
##
```

```
P0AG_associated_genes <- read.delim("EF0_0004190_associations_export.tsv")  
# Rename significant our gene data sets for clarity  
ensemble_genes_6 <- all_genes_6_adj  
ensemble_genes_11 <- all_genes_11_adj  
  
# Convert ensemble gene codes to actual gene names  
gene_names_6 <- AnnotationDbi::select(org.Hs.eg.db, keys = ensemble_genes_6, columns = "SYMBOL", keytype = "ENSEMBL")
```

```
## 'select()' returned 1:1 mapping between keys and columns
```

```
gene_names_11 <- AnnotationDbi::select(org.Hs.eg.db, keys = ensemble_genes_11, columns = "SYMBOL", keytype = "ENSEMBL")
```

```
## 'select()' returned 1:many mapping between keys and columns
```

```
# Get the unique genes from the GWAS study that are significantly differentially expressed at the significance level we are using (5e-10)
significant_GWAS_genes <- POAG_associated_genes[POAG_associated_genes$pValue <= 5e-10, ]
split_genes <- significant_GWAS_genes %>%
  separate_rows(mappedGenes, sep= ",")
split_genes$mappedGenes <- trimws(split_genes$mappedGenes)
unique_genes <- unique(split_genes$mappedGenes)

# Find the overlap between the genes we identified and the genes from the GWAS study
common_genes_6 <- intersect(unique_genes, gene_names_6$SYMBOL)
common_genes_11 <- intersect(unique_genes, gene_names_11$SYMBOL)
common_genes <- intersect(common_genes_11, common_genes_6)
```

Print out the genes:

```
# 6 sample
common_genes_6
```

```
## [1] "FBX032" "KALRN" "FAM13A" "ANGPTL7"
```

```
# 11 sample
common_genes_11
```

```
## [1] "CDKN2B-AS1" "THSD7A" "PDE7B-AS1" "CASC20" "RBM39P1"
## [6] "PLEKHA7" "ZBTB44-DT" "PKHD1" "FBX032" "LINC01873"
## [11] "RARB" "AGBL2" "RPE65" "AC0XL" "KBTBD8"
## [16] "PSMC1P4" "ADAMTS19" "RFX4" "MYO16" "GCAWKR"
## [21] "RUNX1" "DUSP1" "DGKB" "CNBD1" "MECOM"
## [26] "LINC02687" "OR5BA1P" "COL24A1" "PKN2-AS1" "RALGPS1"
## [31] "SLC30A10" "SYN3" "LINC00114" "EGLN3" "MIR9-2HG"
## [36] "MYH13" "FLJ40194" "PCL0" "FAM13A" "LINC01833"
```

```
# Overlap
common_genes
```

```
## [1] "FBX032" "FAM13A"
```